OCEAN: Fast Discovery of High Utility Occupancy Itemsets

Bilong Shen[1], Zhaoduo Wen[2], Ying Zhao[1(✉)], Dongliang Zhou[3], and Weimin Zheng[1]

[1] Department of Computer Science and Technology, Tsinghua Uinversity, Beijing, China
shenbilong@gmail.com, {yingz,zwm-dcs}@tsinghua.edu.cn
[2] School of Information and Telecommunication Engineering, Beijing University of Posts and Telecommunications, Beijing, China
wenzhaoduo@gmail.com
[3] Department of Thermal Engineering, Tsinghua Uinversity, Beijing, China
zhoudongliang08@gmail.com

Abstract. Frequent pattern mining has been widely studied in the past decades and has been applied to many domains. In particular, numerical transaction databases, where not only the items but also the utility associated with them are available in user transactions, are useful for real applications. For example, customer mobile App traffic data collected by mobile service providers contains such information. In this paper, we aim to find frequent patterns that occupy a large portion of total utility of the supporting transactions, to answer questions like "On which mobile Apps do the customers spend most of their data traffic?" Towards this goal, we define a measure called *utility occupancy* to measure the contribution of a pattern within a transaction. The challenge of high utility occupancy itemset discovering is the lack of monotone or anti-monotone property. So we derive an upper bound for *utility occupancy* and design an efficient mining algorithm called OCEAN based on a fast implementation of utility list. Evaluations on real world mobile App traffic data and other three datasets show that OCEAN is efficient and effective in finding frequent patterns with large utility occupancy.

Keywords: Frequent pattern mining · High utility mining · Utility occupancy · Upper bound

1 Introduction

Frequent pattern mining has been extensively studied over decades and has been applied to marketing, recommendation, and other important domains. In addition to the pattern frequency, other properties of patterns, such as utility [4] and occupancy [14], have been studied to meet the needs of various applications and different types of transaction databases. Among them, transaction databases with utility information, which reflects users' interests to items, are

© Springer International Publishing Switzerland 2016
J. Bailey et al. (Eds.): PAKDD 2016, Part I, LNAI 9651, pp. 354–365, 2016.
DOI: 10.1007/978-3-319-31753-3_29

important to study. For example, mobile service providers may collect customer mobile App usage profiles upon approval and use them to improve their service. In this case, each customer mobile App traffic profile (daily or monthly) is a transaction, with a mobile App as an item, and its traffic usage as its utility. Another example can be found in browsing logs, where each website with its browsing time can be viewed as an item and its utility.

In these databases, in order to better capture customer behaviors, *e.g*, answering questions like "On which mobile Apps do the customers spend most of their data traffic?", it is useful to find frequent patterns that are supported by a large portion of transactions and can explain individual customer behaviors to a large extent. In particular, we want to find patterns that occupy a large portion of total utility of the supporting transactions. Towards this goal, we define a novel measure called *utility occupancy* to measure the utility contribution of a pattern within a transaction. Different from existing utility measures, *utility occupancy* uses a relative utility ratio in each transaction to emphasize the importance of a pattern to each customer.

In this paper, we propose a novel frequent pattern mining algorithm that can find frequent patterns with high utility occupancy, and make the following contributions:

1. A novel measure, called *utility occupancy* is proposed to describe the interestingness of a frequent pattern for transactions with utility information.
2. An efficient mining algorithm, called **OCEAN**, is developed. **OCEAN** derive the properties of *utility occupancy* upper bound to pruning the searching space for high efficient pattern mining.
3. Extensive experiments were performed to evaluate the effectiveness and efficiencies of **OCEAN** on various databases, including a real world mobile App traffic usage dataset and three real world datasets that are often used to evaluate high utility mining algorithms.

The rest of this paper is organized as follows. Section 2 discusses some of the related work. Section 3 defines *utility occupancy* and presents the problem formulation. Section 4 derives an upper bound and presents the details of OCEAN. Section 5 provides the detailed experimental evaluation of OCEAN. Finally, Sect. 6 provides some concluding remarks.

2 Related Work

In addition to the pattern frequency, other properties and interestingness measures of frequent patterns and association rules have been studied in the literature [13], to fulfill the need for patterns with good quality, *e.g*, closed frequent patterns [7], maximal frequent patterns [3,15], patterns with constraints [6], and recently, patterns with high occupancy [14], which means the pattern that occupies a large portion of the transactions it appears in. The occupancy of an itemset in a transaction was defined as the ratio between the number of items in the itemset and the number of items in the transaction, and the occupancy

of an itemset in a transaction database was defined as the harmonic average of its occupancy values in all supporting transactions in [14]. According to [14], patterns with high occupancy were more effective in webpage print region recommendation than maximal frequent patterns. Our work is an extension of itemset occupancy in case of transaction databases with utility information. Since we are dealing with a different type of data, the formulation and derivation of the upper bound for *utility occupancy* is tight to the numerical nature of data closely.

Another related problem to our work, which is also an emerging and important topic in marketing, is high utility itemset mining [2,4,5,10,11]. In this problem, each transaction contains the quantity of a set of items, and an item profit (utility) table is also available. The utility of an item in a transaction is defined as the multiplication of its profit and its quantity in the transaction, and the utility of an itemset is defined as the summation of the utilities of the items in the itemset. High utility itemset mining is to find itemsets with high utility or other utility related objectives defined by users. Recent works have been focusing on discussions on different utility functions [8,9], and efficient mining algorithms [2,5,10,11]. For example, HUI-miner [11] and FHM [5] can discover high utility itemsets without candidate generation. Unlike other existing absolute utility measures, the proposed measure *utility occupancy* is a relative measure, which emphasizes whether a certain set of items is important for individual users.

3 Problem Formulation

Suppose a transaction database contains a set of transactions Γ. A *transaction* t containing k items in Γ is in the form of $\{(item_1, util_1), (item_2, util_2), \cdots, (item_k, util_k)\}$, where $item_i$ is the identification of the ith item in t and $util_i$ is a numerical value to represent its utility in t. When a utility table is available, such as shown in Table 1(a), $util_i$ is usually calculated as follows.

Definition 1. *The utility of item i in transaction t, denoted as $U_t(i)$, is the product of the external utility of item i given in a utility table and the count of item i in t. For example, $U_{T2}(c) = 4 \times 3 = 12$ in Table 1.*

Table 1. Database

(a) Utility table

Item	Utility
a	1
b	1
c	4
d	5
e	2
f	3
g	3

(b) Transaction table (item count)

Tid	T1	T2	T3	T4	T5	T6	T7	T8
a	1	9		1		1		
b	1			1	8	2		
c	1	3			2	1		1
d		2	1		2			
e	1	1		1			3	
f		1			1		3	3
g		2	1		1		1	
U(t)	8	42	8	4	32	7	18	13

We say an itemset W is supported by a transaction t, if and only if, for each $item_i$ in W, $item_i$ is also in t. We denote the set of *supporting* transactions of W as Γ_W. The *support ratio* of W, denoted as $\theta(W)$, is calculated as $\frac{|\Gamma_W|}{|\Gamma|}$.

The total utility of a transaction t is defined as the summation of the utilities of all items in t, *i.e*, $U(t) = \sum_{item_i \in t} U_t(item_i)$. For example, the utilities of all transactions in Table 1(b) are shown in the last row. The utility of an itemset W in a transaction t, denoted as $U_t(W)$, is defined as the summation of the utilities of the items present in both t and W, *i.e*, $U_t(W) = \sum_{item_i \in t \cap W} U_t(item_i)$. For example, let $W = \{f, g\}$, $U_{T2}(W) = 9$. The transaction-y utilities of an itemset W in a database Γ, denoted as $TWU(W)$, is defined as the summation of the utilities of W's supporting transactions, *i.e*, $TWU(W) = \sum_{t \in \Gamma_W} U(t)$. For example, let $W = \{f, g\}$, $TWU(W) = U(T2) + U(T5) + U(T7) = 42 + 32 + 18 = 92$.

Now we are ready to define **utility occupancy** for an itemset W.

Definition 2. *The utility occupancy of an itemset W in its supporting transaction t is defined as*

$$\phi_t(W) = \frac{U_t(W)}{U(t)} = \frac{\sum_{item_i \in W, item_i \in t} util_i}{\sum_{item_i \in t} util_i}.$$

The utility occupancy of W in a database is defined as the arithmetic average of the utility occupancy values of W in all its supporting transactions,

$$\phi(W) = \frac{\sum_{t \in \Gamma_W} \frac{U_t(W)}{U(t)}}{|\Gamma_W|}$$

For example, let $W = \{f, g\}$, $\phi_{T2}(W) = 9/42$ and $\phi(W) = (\phi_{T2}(W) + \phi_{T5}(W) + \phi_{T7}(W))/3 = (9/42 + 6/32 + 12/18)/3 = 0.355$.

Definition 3. *Given a minimum support threshold α and a minimum utility occupancy threshold β, the problem of* **high utility occupancy itemset mining** *is to find all itemsets whose support ratio is no less than α and utility occupancy is no less than β.*

Note that the minimum support threshold must be specified, otherwise any transaction itself is a high utility occupancy itemset with a value of 1. Clearly, the utility occupancy measure does not hold downward closure property. In the example of Table 1, $\phi(\{f, g\}) = 0.355$, $\phi(\{d, g\}) = (16/42 + 13/32 + 8/8)/3 = 0.596$, and $\phi(\{d, f, g\}) = (19/42 + 16/32)/2 = 0.476$, which means when an itemset is extended, its utility occupancy can either increase or decrease.

Now we compare high utility occupancy itemsets with high utility itemsets [4] and high occupancy itemsets [14] to have a better understanding of the nature of high utility occupancy itemsets. If we view each transaction as a usage pattern of a user, high utility occupancy itemset mining selects the itemsets that can explain the usage of a group of users to a large extent, whereas high utility itemsets simply represent the ones with highest overall utilities across the database.

For example, compare itemset $\{a, b\}$ with itemset $\{f, g\}$ in our example. Both of them appear in three supporting transactions (T1, T4, and T6 for $\{a, b\}$, T2, T5, and T7 for $\{f, g\}$). $\{f, g\}$ has much larger utility ($U(\{f, g\}) = 9 + 6 + 12 = 27$) than $\{a, b\}$ ($U(\{a, b\}) = 2 + 2 + 3 = 7$), however, the utility occupancy of $\{a, b\}$ ($\phi(\{a, b\}) = (2/8 + 2/4 + 3/7)/3 = 0.392$) is higher than that of $\{f, g\}$ ($\phi(\{f, g\}) = 0.355$). Our proposed high utility occupancy itemset mining is an extension of the high occupancy itemset mining problem proposed in [14], which defines occupancy using the cardinality of an itemset instead of utility. For example, in transaction T6 of our example, the occupancy of $\{a, b\}$ is higher than that of $\{c\}$ in terms of cardinality, however, $\phi_{T6}(\{c\}) = 4/7 > \phi_{T6}(\{a, b\}) = 3/7$.

4 High Utility Occupancy Itemset Mining

In this section, we first derive an upper bound for utility occupancy, and then describe an efficient algorithm **OCEAN** to find qualified patterns with the help of the derived upper bound.

4.1 Upper Bound of Utility Occupancy

The process of itemset mining can be viewed as an exploration of a set enumeration tree [12], whose root node is an empty set, and each node represents an itemset and its children are extended itemsets from it by a certain order. The order can be lexicographic order, or transaction-weighted utility ascending order, which is a common choice in many high utility itemset mining algorithms. In our example, according to the values of transaction-weighted utility, we have $b \prec a \prec e \prec d \prec g \prec c \prec f$, and we use this order for the rest of the paper. Given a subtree rooted at itemset X, the set of all possible items that can be added to form itemsets in this subtree is called the *extension set* E. For example, let us consider the subtree rooted at g for our example in Fig. 1, and $E = \{c, f\}$ for the node g.

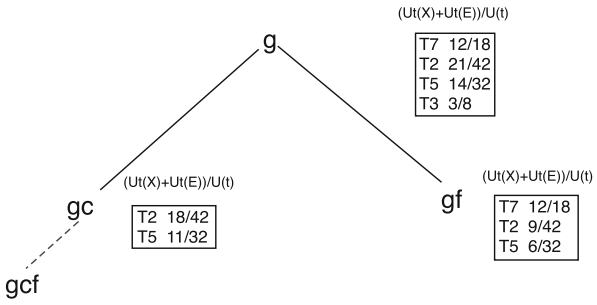


Fig. 1. The subtree rooted at g

Given a subtree rooted at X, we need to derive an upper bound $\hat{\phi}()$ of utility occupancy for all possible qualified patterns residing in the subtree without

actually generating these patterns. If $\hat{\phi}(X)$ is less than the minimum utility occupancy threshold, we can safely discard this subtree from further consideration.

Lemma 1. *Given a subtree rooted at X, its supporting transaction set Γ_X and its extension set E, for any possible qualified itemset W in the subtree, we have*

$$\phi(W) \leq \frac{\sum_{t \in \Gamma_W} \frac{U_t(X)+U_t(E)}{U(t)}}{|\Gamma_W|}.$$

Proof. Since $W - X \subset E$, we have $U_t(W - X) \leq U_t(E)$. Therefore,

$$\phi(W) = \frac{\sum_{t \in \Gamma_W} \frac{U_t(W)}{U(t)}}{|\Gamma_W|} = \frac{\sum_{t \in \Gamma_W} \frac{U_t(X)+U_t(W-X)}{U(t)}}{|\Gamma_W|} \leq \frac{\sum_{t \in \Gamma_W} \frac{U_t(X)+U_t(E)}{U(t)}}{|\Gamma_W|}.$$

□

Theorem 1. *Given the minimum support ratio α, a subtree rooted at X, its supporting transaction set Γ_X and extension set E, for any possible qualified itemset W in the subtree, $\hat{\phi}(W) = \frac{\sum_{top\alpha|\Gamma|, t \in \Gamma_X} (\frac{U_t(X)+U_t(E)}{U(t)})^{\downarrow}}{\alpha|\Gamma|} \geq \phi(W)$.*

Proof. Since Γ_W is unknown, we calculate $\frac{U_t(X)+U_t(E)}{U(t)}$ for all transactions in Γ_X, and sort this vector in descending order, denoted as $(\frac{U_t(X)+U_t(E)}{U(t)})^{\downarrow}$. The average of top $|\Gamma_W|$ values of this vector is an upper bound of $\frac{\sum_{t \in \Gamma_W} \frac{U_t(X)+U_t(E)}{U(t)}}{|\Gamma_W|}$. Furthermore, since a qualified pattern should be supported by at least $\alpha|\Gamma|$ transactions, we have $\alpha|\Gamma| \leq |\Gamma_W|$. Put all together, we have

$$\begin{aligned}\phi(W) &\leq \frac{\sum_{t \in \Gamma_W} \frac{U_t(X)+U_t(E)}{U(t)}}{|\Gamma_W|} \leq \frac{\sum_{top|\Gamma_W|, t \in \Gamma_X} (\frac{U_t(X)+U_t(E)}{U(t)})^{\downarrow}}{|\Gamma_W|} \\ &\leq \frac{\sum_{top\alpha|\Gamma|, t \in \Gamma_X} (\frac{U_t(X)+U_t(E)}{U(t)})^{\downarrow}}{\alpha|\Gamma|}.\end{aligned}$$

□

For example, the sorted $\frac{U_t(X)+U_t(E)}{U(t)}$ vectors are shown for each node in Fig. 1. Suppose the minimum support ratio is 0.25. We can average the top two values from the sorted vectors to calculate $\hat{\phi}()$ for each node. To be specific, $\hat{\phi}(\{g\}) = (12/18 + 21/42)/2 = 0.584$. $\hat{\phi}(\{g, c\}) = (18/42 + 11/32)/2 = 0.387$. If the minimum utility occupancy ratio is 0.5, we can safely prune the subtree rooted at $\{g, c\}$.

4.2 Design and Implementation of OCEAN

We design and implement our proposed high utility occupancy mining algorithm based on an important data structure *utility list*, which was used in the state-of-the-art high utility itemset mining algorithms: HUI-miner [11] and FHM [5].

Algorithm 1. UpperBound

Input: P_X : the root itemset
$minSupp$: the minimum support threshold
Output: the upper bound of utility occupancy for the subtree rooted at P_X
1: Derive $\frac{U_t(X)+U_t(E)}{U(t)}$ from $P_X.UL$ as V;
2: $V_{top} \leftarrow$ the largest $minSupp \times |\Gamma|$ values in V;
3: $sum \leftarrow 0$;
4: **for** $u \leftarrow 1$ **to** $minSupp \times |\Gamma|$ **do**
5: $sum \leftarrow sum + V_{top}(u)$
6: **end for**
7: $occu \leftarrow sum/(minSupp \times |\Gamma|)$;
8: **return** $occu$

For any given itemset X and its extension set E, its utility list is a list of utility records in its supporting transactions. For each supporting transaction t, the utility list contains a three-tuple record, *i.e*, ($transaction_id$, $U_t(X)$, $U_t(E)$). Therefore, the itemset data structure in our algorithms has two fields, *itemset* for the set of items, and UL for its utility list. In both HUI-miner [11] and FHM [5], the direct summation of $U_t(X)$ and $U_t(E)$ for each itemset was used for utiltiy prunning, whereas in OCEAN the upper bound of occupancy is more complicated to calculate, which is given in Algorithm 1. Note that we do not have to sort the entire $\frac{U_t(X)+U_t(E)}{U(t)}$ vector to get the top $minSupp \times |\Gamma|$ values.

The utility records are sorted according to $transaction_id$ in all utility lists, so that the merge of two utility lists can be done efficiently. Given two itemsets P_X and P_Y, suppose $P_X.itemset = X$, $P_Y.itemset = Y$, and $X \cap Y = \varnothing$, the itemset $X \cup Y$ and its utility list can be easily constructed by scanning $P_X.UL$ and $P_Y.UL$ simultaneously and adding a utility record for each common transaction t into $P_{XY}.UL$. We name this procedure as $merge(P_X.UL, P_Y.UL)$ and omitted its detailed implementation. In addition, $Size(P_X.UL)$ returns the number of transaction records in $P_X.UL$.

The main idea of our proposed algorithm OCEAN is to perform a depth first search on the set enumeration tree of a database. In Algorithm 2, for each item i, we create an itemset I_i such that $I_i.itemset = \{i\}$ and $I_i.UL$ contains its utility list. I is a set of such itemsets for all items and is sorted in ascending order according to the transaction-weighted utility of each item. Such order is also used in the depth first search, which is implemented in Algorithm 3. In Algorithm 3, qualified itemsets are reported (in line 2 and 3) and node explorations are coordinated by determining whether the node should be pruned (in line 5 and 6) and preparing the set of extension itemsets of the node (in line 8 to 12) for its exploration (in line 13). To be specific, given an itemset X and its extension set E, for each item $y \in E$, $X \cup y$ forms an extension itemset of X.

Algorithm 2. OCEAN

Input:
D: a transaction database
$minOccu$: the minimum utility occupancy threshold
$minSupp$: the minimum support threshold

Output:
The set of itemsets with utility occupancy $\geq minOccu$ and support $\geq minSupp$.

1: Scan $\boldsymbol{D}$ to build the utility list of each item $\boldsymbol{i}$ to form $\boldsymbol{I}$ and calculate transaction utilities and item utilities.
2: **sort** I in ascending order according to the transaction-weighted utility of each item.
3: **Prune**($\varnothing$, I, $minOccu$, $minSupp$)

Algorithm 3. Prune

Input: P : the itemset
ExP : the set of extension itemsets of P
$minOccu$: the minimum utility occupancy threshold
$minSupp$: the minimum support threshold

Output:
The set of itemsets with utility occupancy $\geq minOccu$ and support $\geq minSupp$.

1: **for** each itemset $P_X \in ExP$ **do**
2: **if** $Occu(P_X.UL) \geq minOccu \wedge Size(P_X.UL) \geq minSupp \times |\Gamma|$ **then**
3: Output P_x
4: **end if**
5: $maxOccu \leftarrow UpperBound(P_X)$
6: **if** $maxOccu \geq minOccu \wedge Size(P_X.UL) \geq minSupp \times |\Gamma|$ **then**
7: $ExP_X \leftarrow null$;
8: **for** each item y such that $y \succ X$ **do**
9: $P_{Xy}.itemset \leftarrow P_X.itemset \cup \{y\}$
10: $P_{Xy}.UL \leftarrow merge(P_X.UL, I_y.UL)$
11: $ExP_X \leftarrow ExP_X \cup P_{Xy}$
12: **end for**
13: $Prune(P_X, ExP_X, minOccu, minSupp)$
14: **end if**
15: **end for**

5 Experiment

We evaluated the effectiveness and efficiency of the proposed algorithm OCEAN using four real world datasets. In addition to OCEAN, we also implemented a variation: OCEAN without minOccu pruning as our baseline. All algorithms were implemented in JAVA. Experiments were performed on a PC with a 3.4 Hz Intel i7 processor and 8 GB memory running Windows7 OS.

The four datasets used in our experiments represent various applications, itemset distributions, and other characteristics, as shown in Table 2 including the number of transactions, the number of distinct items, the average length of transactions, and the number of items in the longest transaction(s).

Table 2. Characteristics of datasets

Database	Trans	Items	Avelen	Maxlen	Type
MobileApp	109774	14	3.82	13	dense
Mushroom	8124	119	23	23	dense
Kosarak	990002	41270	8.1	2498	sparse
BMS	59602	497	2.5	267	sparse

MobileApp was obtained from a mobile service provider, with each transaction recording the traffic data that a customer spends on various mobile Apps. **Mushroom**, **BMS**, and **Kosarak** are benchmark datasets for high utility itemset mining (downloaded from FIMI Repository [1]). Mushroom includes descriptions of hypothetical samples corresponding to 23 species of gilled mushrooms, each transaction contains 23 items. BMS is a real-life sparse click-stream data from a webstore. Kosarak is a real-life sparse click-stream data of a hungarian on-line news portal. Unlike MobileApp, the other three datasets provide transactions without utility information. So we generated the external utilities for items between 1 and 10 by using a log-normal distribution and randomly generated item counts between 1 and 5 as in [5,11].

5.1 High Utility Occupancy Itemsets Vs. High Utility Itemsets

We used MobileApp to illustrate the effectiveness of high utility occupancy itemsets for identifying the set of mobile Apps that users are interested in. Towards this goal, we used OCEAN and FHM to generate 10 itemsets with highest utility occupancy values and highest utilities, respectively. The results are shown in Table 3. The itemsets found by OCEAN are more favorable than those found by FHM for two reasons. First, high utility occupancy itemsets indicate high interests of users in these itemsets. For example, itemset {7,9,14}, as shown in Table 3, has a relatively low utility of 18.1 %, but it has a high utility occupancy of 77.4 %, which means customers who use mobile Apps {7,9,14} spend 77.4 % of their traffic quota on these applications on average. Second, the size of itemsets found by OCEAN is larger than that by FHM in general, because larger itemsets lead to a decrease in support ratio which usually harms utility values as well. The diversity of found mobile Apps groups with high occupancy is more favorable for the task of mobile App recommendation or promotion.

5.2 Efficiency of OCEAN

We evaluated the efficiency of OCEAN on BMS, Kosarak, and Mushroom datasets. We compared the running time and the number of searched itemsets for OCEAN and OCEAN without minOccu pruning, shorted as OCEAN_no, to see how effective our derived upperbound is. The results are shown in Fig. 2. We fixed minOccu and varied minSupp in Fig. 2(a), (b) and (c), and we fixed

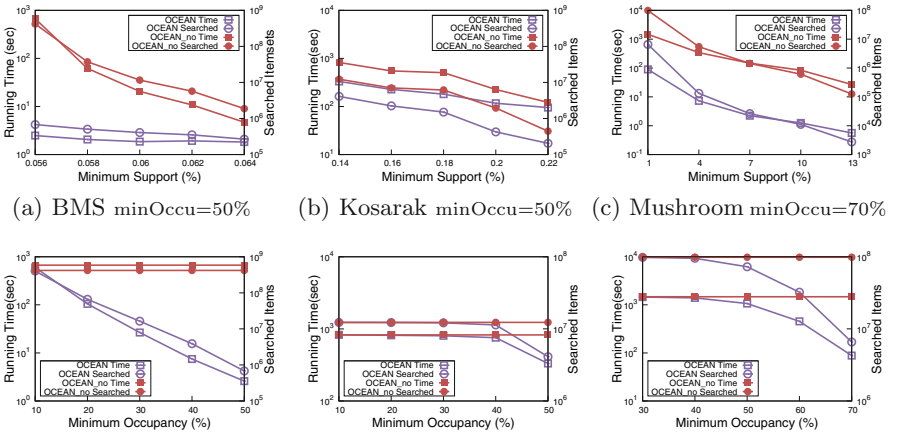
Table 3. Top 10 results of MobileApp

(a) OCEAN

Pattern	Occu(%)	Util(%)	Supp(%)
{7,9,14}	77.4	18.1	21.8
{2,9,10}	74.5	16.4	14.1
{7,10,14}	73.7	13.3	16.1
{7,9,10}	73.6	21.4	19.0
{7,14}	72.7	35.2	65.7
{9,14}	71.6	19.3	23.1
{9,10}	71.4	25.3	22.6
{10,14}	66.8	14.0	17.6
{8,9}	65.0	14.2	12.8
{14}	64.4	35.7	68.9

(b) FHM

Pattern	Occu(%)	Util(%)	Supp(%)
{14}	64.4	35.7	68.9
{7,14}	72.7	35.2	65.7
{9}	42.4	25.4	44.3
{9,10}	71.4	25.3	22.6
{7,9}	45.6	21.8	38.1
{7,9,10}	73.6	21.4	19.0
{9,14}	71.6	19.3	23.1
{10}	43.8	19.1	38.1
{7}	77.4	18.8	41.4
{7,9,14}	77.4	18.1	21.8

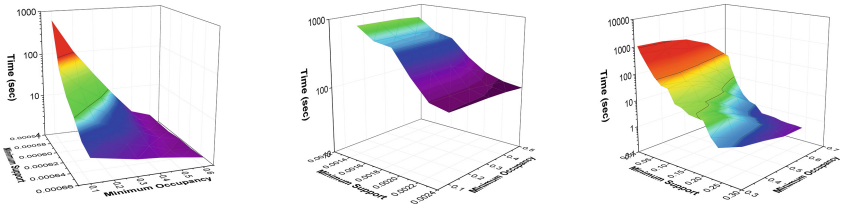


(a) BMS minOccu=50% (b) Kosarak minOccu=50% (c) Mushroom minOccu=70%

(d) BMS minSupp=0.056% (e) Kosarak minSupp=0.14% (f) Mushroom minSupp=1%

Fig. 2. Running time and number of searched itemsets by OCEAN and OCEAN_no

minSupp and varied minOccu in Fig. 2(d), (e) and (f). Clearly, with increasing minOccu, OCEAN pruned more itemsets in addition and run in less time. In general, the running time and the number of searched itemsets of OCEAN decreased as minOccu and minSupp increased. For sparse datasets, small minSupp and minOccu can be effective too. For example, there are 41270 distinct items in Kosarak. With minSupp and minOccu being 0.14 %, 10 % respectively, OCEAN actually searched 12290355 itemsets, whereas there are 2^{41270} possible itemsets in total. We can see the trend more clearly in Fig. 3, in which we measured the running time of OCEAN with varying minSupp and minOccu. The higher the minSupp or minOccu is, the fewer the number of searched itemsets is and thus the less running time is.



(a) BMS (b) Kosarak (c) Mushroom

Fig. 3. Running time by OCEAN

We also studied the memory consumption of OCEAN. We found that for all databases we used, the memory consumption is stable. For example, for Mushroom, the consumption is almost 161 MB for all minSupp and minOccu values. We omitted the detailed memory consumption results due to space constraint.

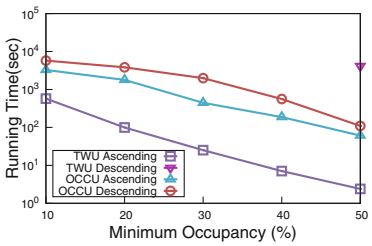


Fig. 4. Running time by OCEAN with various item orders on BMS

The efficiency of OCEAN is manifestly influenced by the processing order of items. We recorded the running time of OCEAN using TWU (transaction-weighted utility) ascending order, TWU descending order, item occupancy ascending order, item occupancy descending order, and lexicographic order on BMS. Figure 4 shows the experimental results. According to Fig. 4, the TWU ascending order made a best performance as it can reduce the number of generated utility lists [11], thus OCEAN processes items by following this order. Note that the curve of lexicographic order is not shown in Fig. 4 since no results were generated under 1×10^5 s.

6 Conclusion

In this paper, we propose *utility occupancy* as a measure for interestingness of itemsets with utility information.The proposed measure can lead to useful itemsets that contribute a large portion of total utility for each individual transaction representing user interests or user habit. We derive an upper bound for utility

occupancy from an itemset for all possible extension itemsets from it, and design an efficient mining algorithm using this upper bound based on a utility list data structure and a depth first search procedure. Experimental evaluations on real world data show that the proposed mining algorithm can find such patterns effectively and efficiently.

References

1. Frequent itemset mining implementations repository. http://fimi.ua.ac.be/data/
2. Ahmed, C.F., Tanbeer, S.K., Jeong, B.S., Lee, Y.K.: Efficient tree structures for high utility pattern mining in incremental databases. IEEE Trans. Knowl. Data Eng. **21**(12), 1708–1721 (2009)
3. Burdick, D., Calimlim, M., Gehrke, J.: Mafia: A maximal frequent itemset algorithm for transactional databases. In: Proceedings of the 17th International Conference on Data Engineering (ICDE), pp. 443–452. IEEE (2001)
4. Chan, R.C., Yang, Q., Shen, Y.D.: Mining high utility itemsets. In: Proceedings of the 3rd IEEE International Conference on Data Mining (ICDM), pp. 19–26. IEEE (2003)
5. Fournier-Viger, P., Wu, C.-W., Zida, S., Tseng, V.S.: FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In: Andreasen, T., Christiansen, H., Cubero, J.-C., Raś, Z.W. (eds.) ISMIS 2014. LNCS, vol. 8502, pp. 83–92. Springer, Heidelberg (2014)
6. Han, J., Cheng, H., Xin, D., Yan, X.: Frequent pattern mining: Current status and future directions. Data Min. Knowl. Disc. **15**(1), 55–86 (2007)
7. Han, J., Wang, J., Lu, Y., Tzvetkov, P.: Mining top-k frequent closed patterns without minimum support. In: Proceedings of the 2rd IEEE International Conference on Data Mining (ICDM), pp. 211–218. IEEE (2002)
8. Hong, T.P., Lee, C.H., Wang, S.L.: Effective utility mining with the measure of average utility. Expert Syst. Appl. **38**(7), 8259–8265 (2011)
9. Lan, G.C., Hong, T.P., Tseng, V.S.: A projection-based approach for discovering high average-utility itemsets. J. Inf. Sci. Eng. **28**(1), 193–209 (2012)
10. Liu, J., Wang, K., Fung, B.: Direct discovery of high utility itemsets without candidate generation. In: Proceedings of the 12th IEEE International Conference on Data Mining (ICDM), pp. 984–989. IEEE (2012)
11. Liu, M., Qu, J.: Mining high utility itemsets without candidate generation. In: Proceedings of the 21st ACM International Conference on InformatIon and Knowledge Management (CIKM), pp. 55–64. ACM (2012)
12. Rymon, R.: Search through systematic set enumeration. Technical Reports (CIS), pp. 539–550 (1992)
13. Tan, P.N., Kumar, V., Srivastava, J.: Selecting the right objective measure for association analysis. Inf. Syst. **29**(4), 293–313 (2004)
14. Tang, L., Zhang, L., Luo, P., Wang, M.: Incorporating occupancy into frequent pattern mining for high quality pattern recommendation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM), pp. 75–84. ACM (2012)
15. Yang, G.: The complexity of mining maximal frequent itemsets and maximal frequent patterns. In: Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 344–353. ACM (2004)